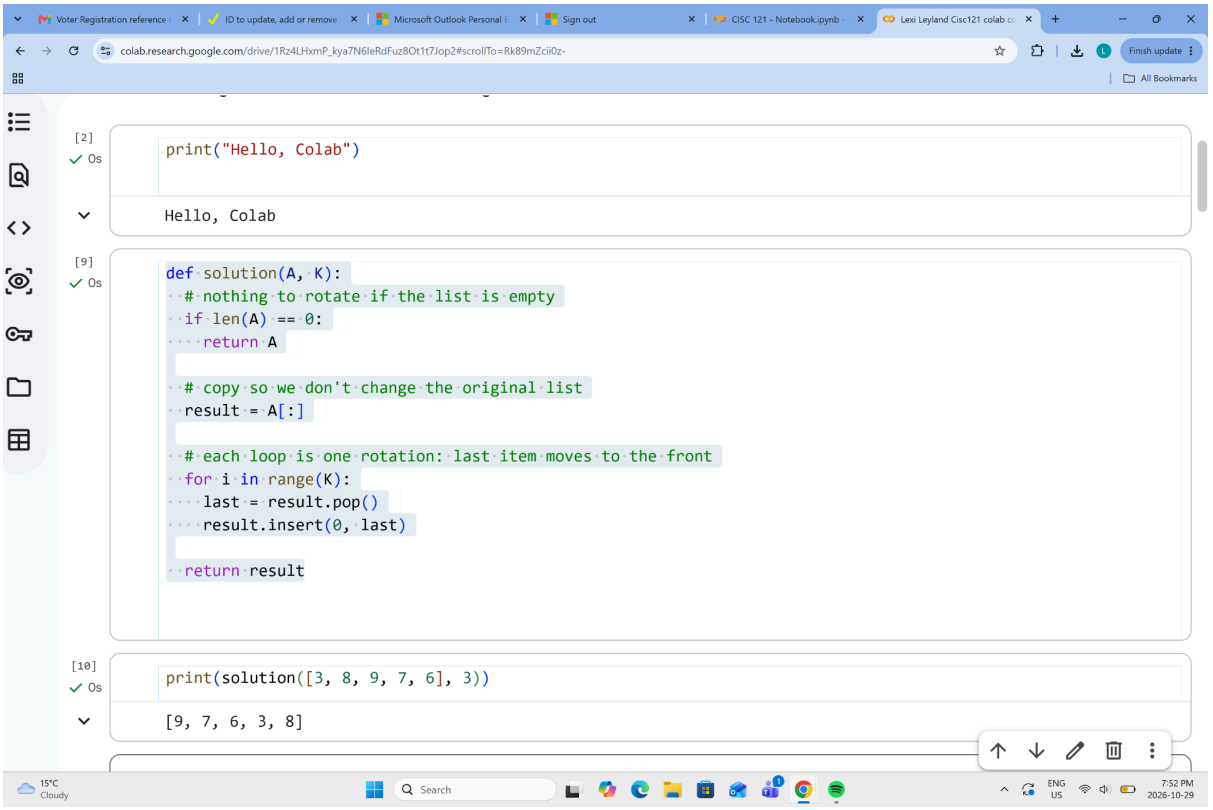


```
print("Hello, Colab")
```

```
Hello, Colab
```

```
def solution(A, K):
    # nothing to rotate if the list is empty
    if len(A) == 0:
        return A

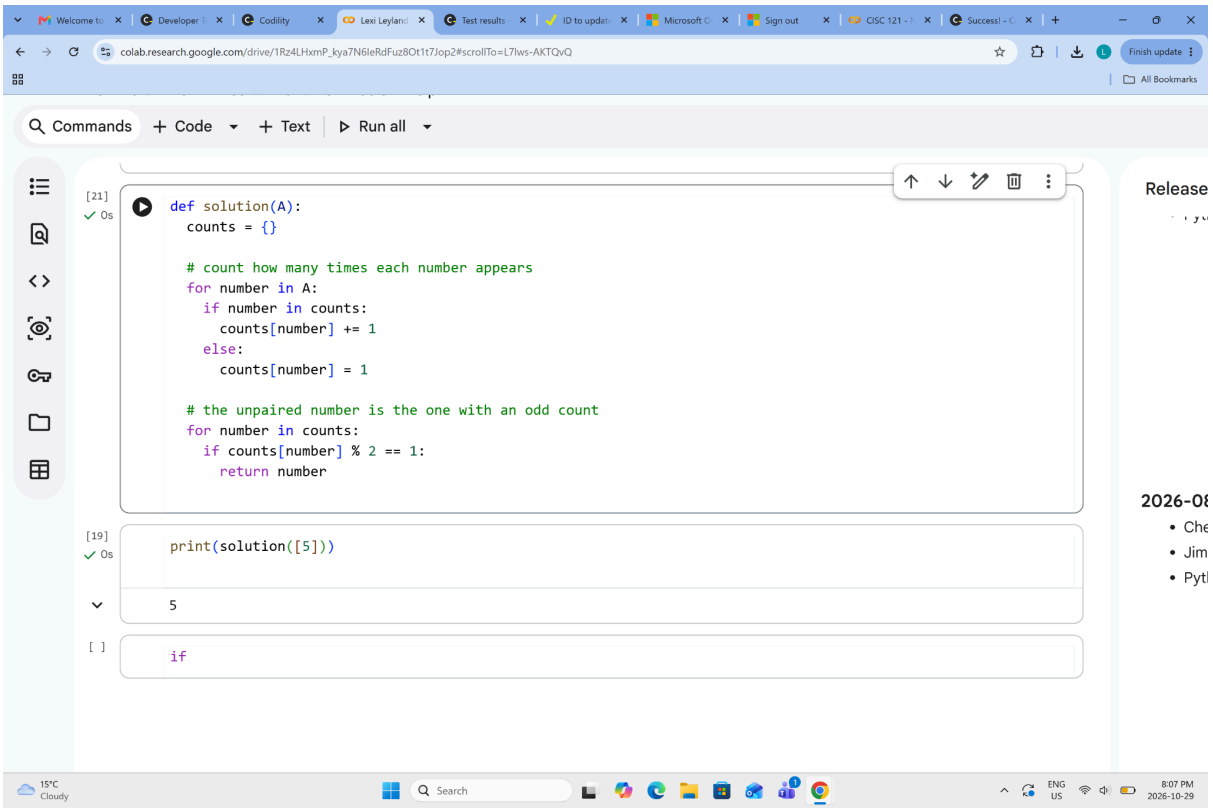
    # copy so we don't change the original list
    result = A[:]

    # each loop is one rotation: last item moves to the front
    for i in range(K):
        last = result.pop()
        result.insert(0, last)

    return result
```

```
print(solution([3, 8, 9, 7, 6], 3))
```

```
[9, 7, 6, 3, 8]
```



```
def solution(A):
  counts = {}

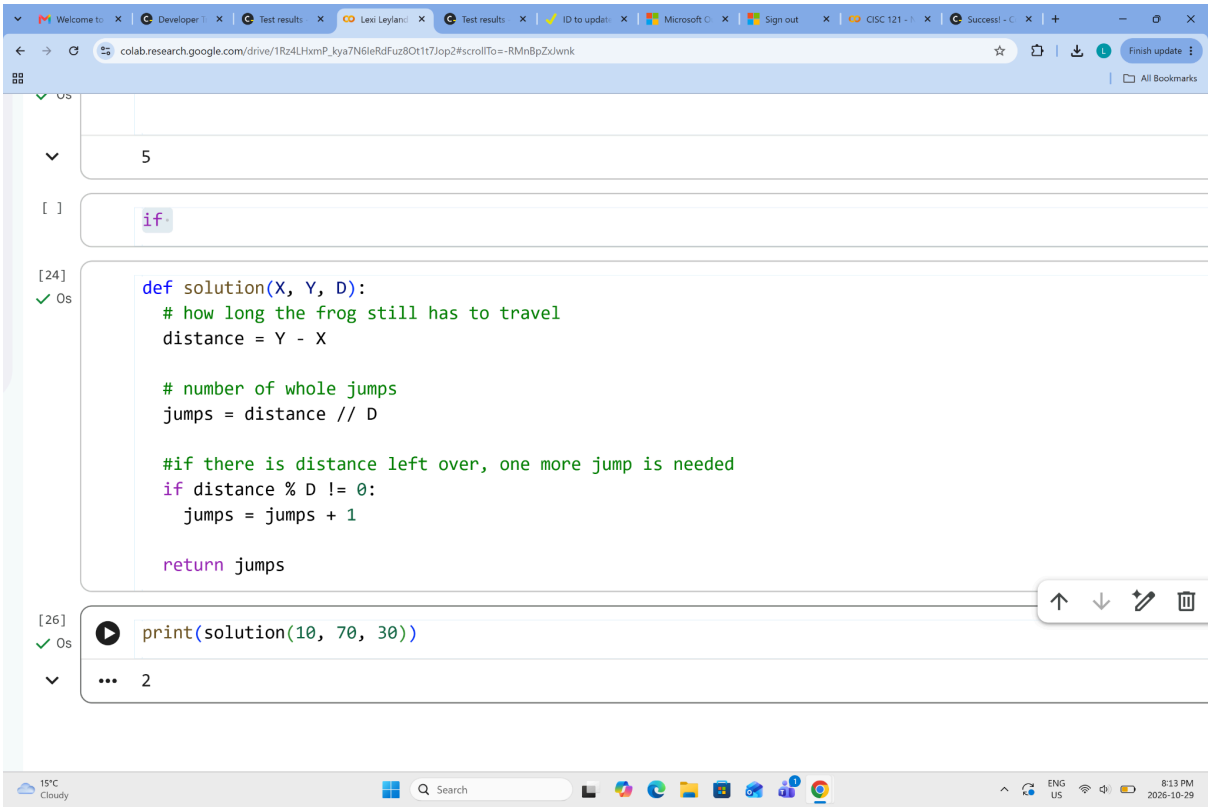
  # count how many times each number appears
  for number in A:
    if number in counts:
      counts[number] += 1
    else:
      counts[number] = 1

  # the unpaired number is the one with an odd count
  for number in counts:
    if counts[number] % 2 == 1:
      return number
```

```
print(solution([5]))
```

```
5
```

```
if
```



```
5
```

```
[ ]
if
```

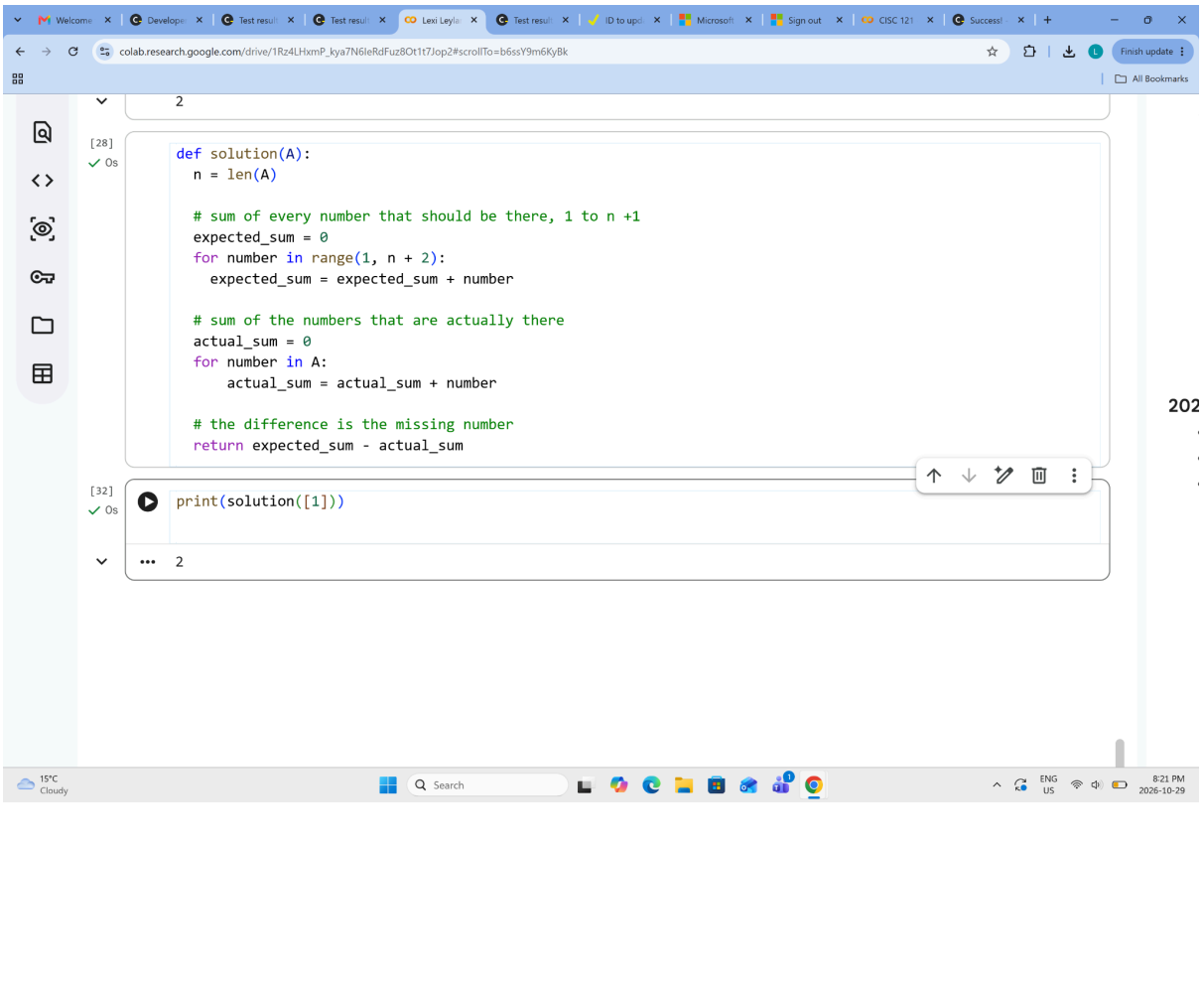
```
[24]
✓ 0s
def solution(X, Y, D):
  # how long the frog still has to travel
  distance = Y - X

  # number of whole jumps
  jumps = distance // D

  #if there is distance left over, one more jump is needed
  if distance % D != 0:
    jumps = jumps + 1

  return jumps
```

```
[26]
✓ 0s
print(solution(10, 70, 30))
...
2
```



```
2
```

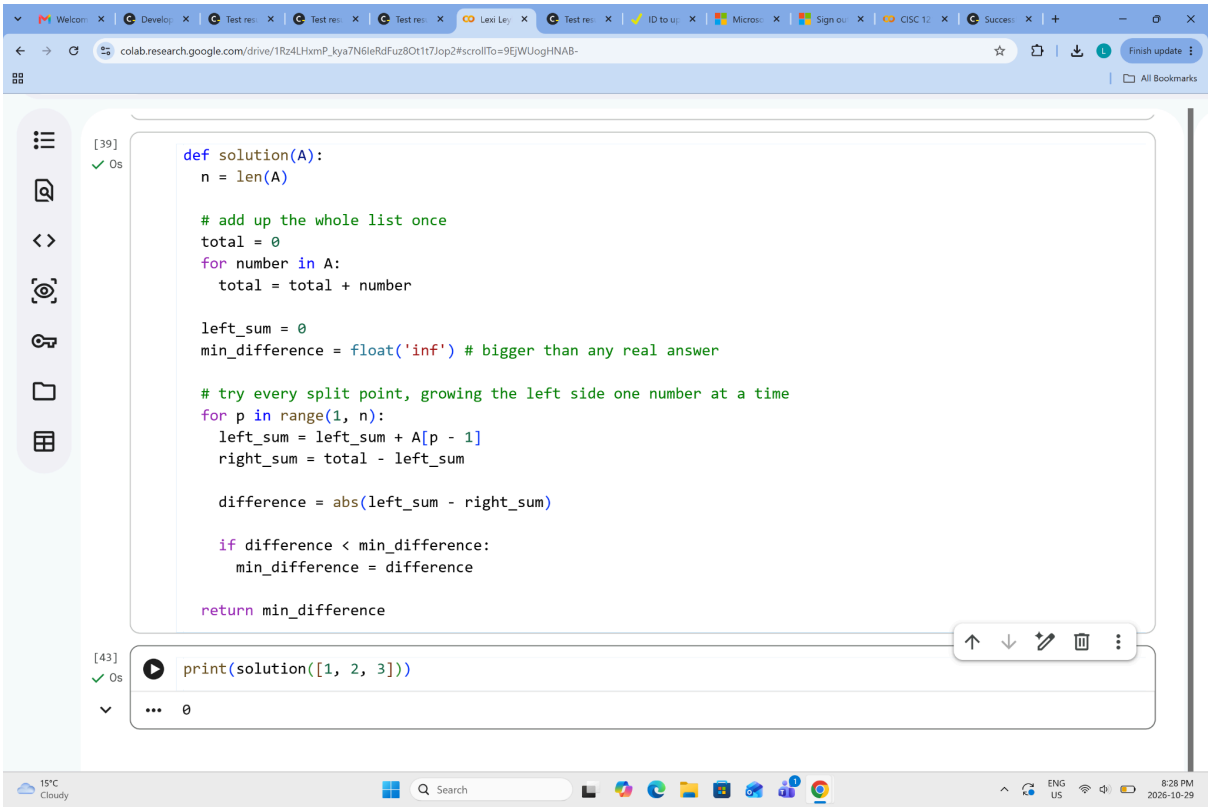
```
[28]
✓ 0s
def solution(A):
  n = len(A)

  # sum of every number that should be there, 1 to n +1
  expected_sum = 0
  for number in range(1, n + 2):
    expected_sum = expected_sum + number

  # sum of the numbers that are actually there
  actual_sum = 0
  for number in A:
      actual_sum = actual_sum + number

  # the difference is the missing number
  return expected_sum - actual_sum
```

```
[32]
✓ 0s
print(solution([1]))
...
2
```



```
[39]
✓ 0s
def solution(A):
  n = len(A)

  # add up the whole list once
  total = 0
  for number in A:
    total = total + number

  left_sum = 0
  min_difference = float('inf') # bigger than any real answer

  # try every split point, growing the left side one number at a time
  for p in range(1, n):
    left_sum = left_sum + A[p - 1]
    right_sum = total - left_sum

    difference = abs(left_sum - right_sum)

    if difference < min_difference:
      min_difference = difference

  return min_difference
```

```
[43]
✓ 0s
print(solution([1, 2, 3]))
...  0
```